

Table of contents

Overview of the system	2
Overview of the application and it's components	2
Overview of the applications deployment process.....	4
How to test the system	6
Implemented optional features	6
General instructions.....	6
Testing the application from browser.....	6
Testing the application using API.....	7
Development environment.....	7
Example runs	8
Reflections	11
Main learnings and worst difficulties.....	11
Amount of effort (hours) used	11
Usage of LLM	11

Overview of the system

Overview of the application and it's components

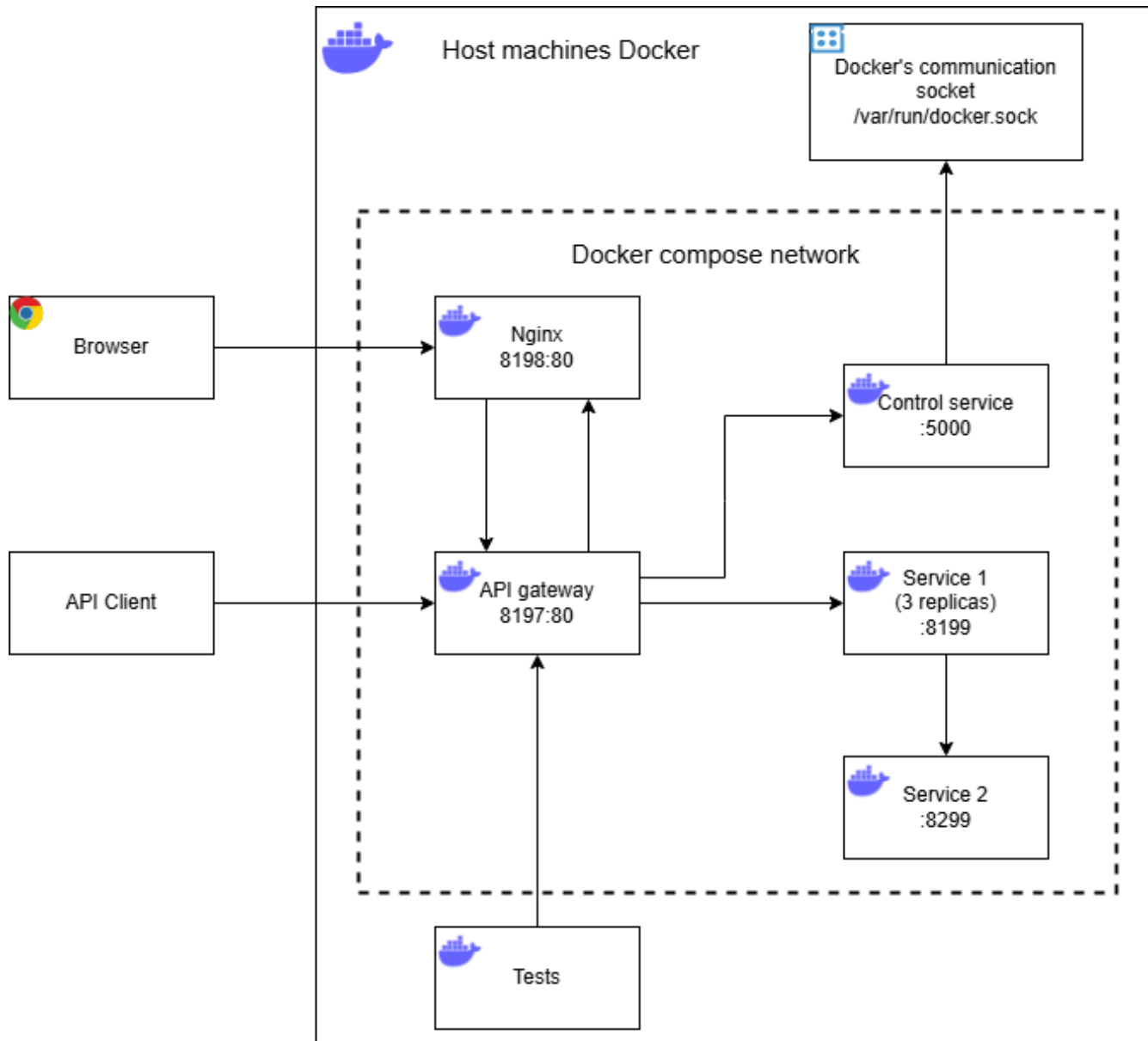


Diagram 1: Overview of the application components when the application is running

The table below explains each component and it's purpose briefly

Component	Purpose	(Programming) languages, libraries, etc.
Nginx	Acts as an HTTP server that browser uses to communicate with. Delivers the frontend application (index.html) to the browser. Redirects requests from browser to API gateway. Responsible for authorization.	Nginx configuration, HTML/JavaScript
API gateway	Acts as an RestAPI for incoming requests both from Nginx and external API clients. Responsible for the state of the application. Communicates with Nginx to ensure the user is authorized, if a request requires authorization (changing state from INIT to RUNNING requires authorization). Responsible for routing users request to correct service (service 1 or control service)	NodeJS
Control service	Communicates with the host machines docker socket to shut down all the containers (including it's own container) when asked for.	NodeJS
Service 1	Responds to requests about asking information about the containers state (ip address, hostname, ...). Requests information from service 2.	NodeJS
Service 2	Responds to requests about asking information about the containers state (ip address, hostname, ...).	Python
Tests	Runs automated (unit) tests towards the application. The tests container is built and run separately. There are 13 unit tests that mainly test the functionality from the API gateways point-of-view.	NodeJS, Mocha, Chai

Overview of the applications deployment process

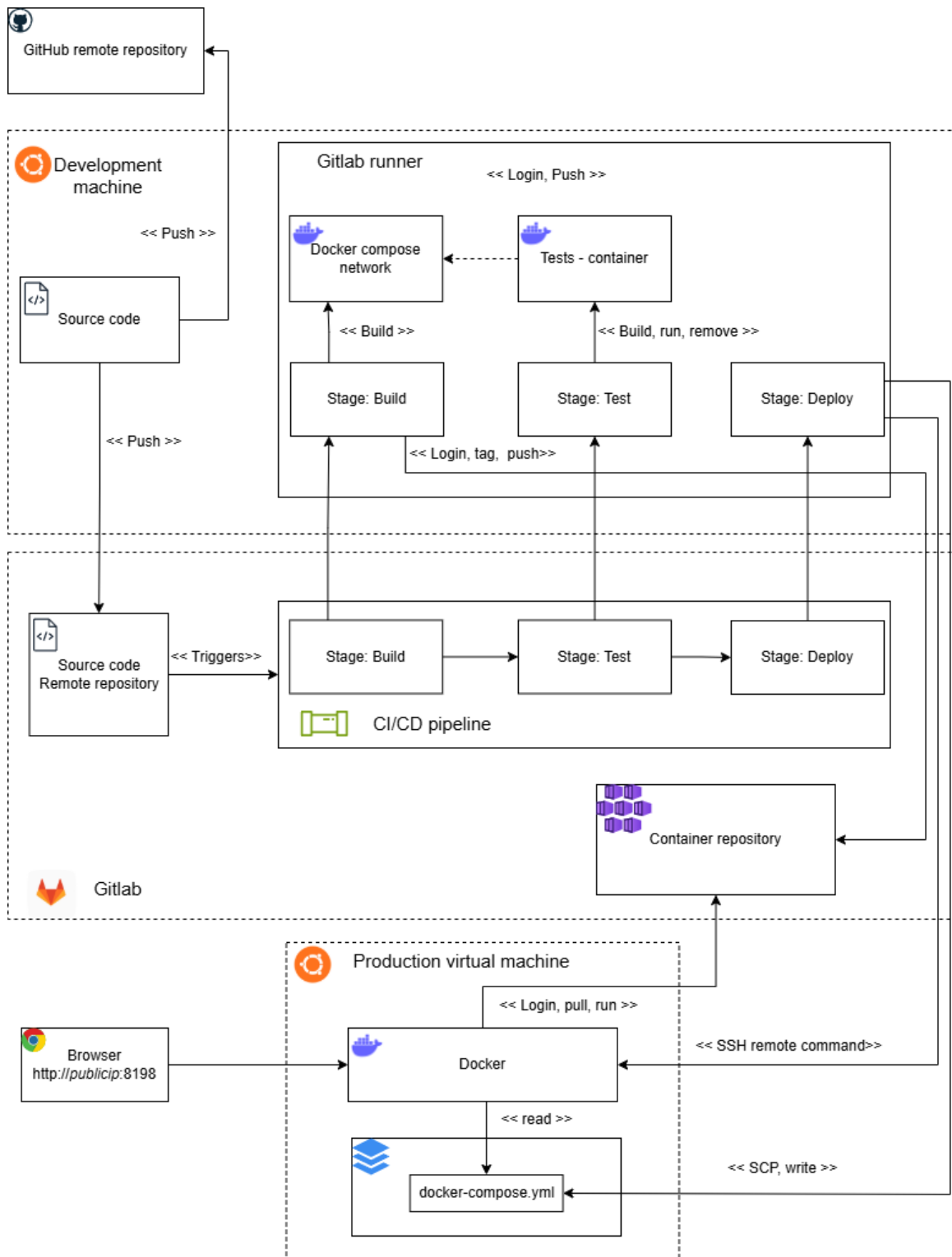


Diagram 2: The deployment process of the application

There are 3 major components that play a crucial role in the deployment: Development machine, Gitlab and Production virtual machine.

The deployment process starts when the source code change is pushed to Gitlab. In the diagram above, the source code change is done from the same development machine where Gitlab runner is running, but naturally the source code change can be pushed from anywhere else.

When a source code change is pushed to Gitlab, Gitlab triggers a CI/CD pipeline. The stages of the pipeline are defined in the source codes `.gitlab-ci.yml` -file. The actual execution of the pipeline steps is done by the Gitlab runner that is installed in the development machine. The CI/CD pipeline in gitlab triggers the runner.

First stage of the pipeline is “build” stage. Gitlab runner builds the application’s containers locally. Containers are pushed to Gitlab’s container registry, so that the production machine can later pull the correct images. Naturally if building the application fails, the pipeline is stopped.

After successful “build” stage, “test” stage is executed. Gitlab runner runs the application locally (which was built in the previous step). Once the application is running, the runner builds a separate “Tests” -container and runs said container. The “tests” container includes testing application that runs unit tests towards the actual application. Assuming that building the application was successful, and the all the tests passed, we continue to “deploy” stage.

A separate dedicated virtual machine has been set up to act as a production environment. The first step of the deploy stage is to deliver the latest `docker-compose.yml` -file to the production machine. This is done by the gitlab runner using SCP. After the latest `docker-compose.yml` has been delivered, the gitlab runner instructs the production machine to pull the deployed container images and start the application. The necessary commands are delivered to production machine using SSH.

Once the production virtual machine has successfully pulled the container images and started the application, the environment can be accessed by browser from anywhere.

How to test the system

Implemented optional features

Deployment to external server -optional feature was implemented.

The application can be accessed by browser: <http://128.214.254.140:8198/> (username: admin, password: admin)

General instructions

Source code can be pulled from Github using:

```
git clone -b project https://github.com/taskijanne/COMP.SE.140_excercise_1.git
```

Once the source code has been cloned, the application can be started (in the root folder of the source code):

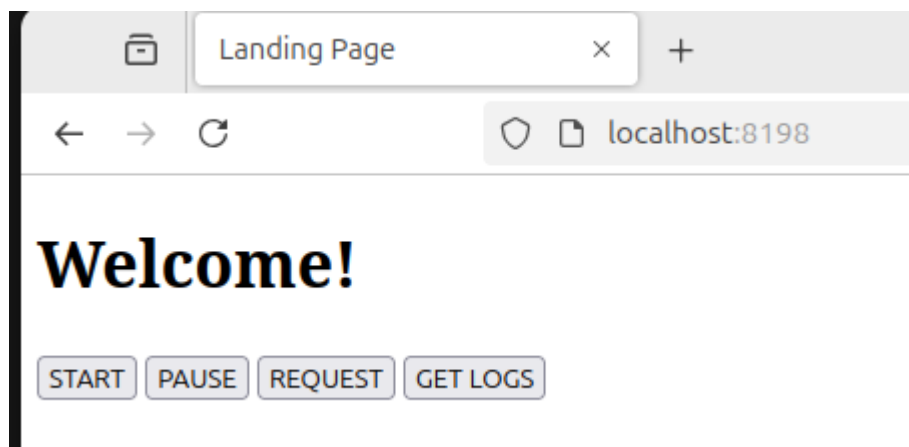
```
docker-compose up -d
```

Testing the application from browser

Once the application is running, open browser with url: <http://localhost:8198>

Username: admin, password: admin

Fancy UI of the application opens:



START: Sets the applications stage to “RUNNING” (by calling PUT /state with payload “RUNNING”)

PAUSE: Sets the applications stage to “PAUSED” (by calling PUT /state with payload “PAUSED”)

REQUEST: Requests data from the services (by calling GET /request)

GET LOGS: Requests logs from the application (by calling GET /run-log)

Important: Once the user has logged in, the state of the application is not changed to “RUNNING” automatically. User must press the “START” button (or call the PUT /state endpoint with “RUNNING” payload with proper authorization header)

Also note that user can try to do “illegal actions” from UI, for example trying to pause the application even when the application is in init -state. Doing this leads to error message (and status code 400).

Testing the application using API

Example commands after the application is running (assuming user hasn’t logged in and changed the state to RUNNING):

Authenticate and set the status “Running”:

```
curl localhost:8197/state -X PUT -d "RUNNING" -H "Content-Type:text/plain" -H  
"Authorization:Basic YWRtaW46YWRtaW4="
```

Switch status to “Paused” (no need to authenticate anymore):

```
curl localhost:8197/state -X PUT -d "PAUSED" -H "Content-Type:text/plain"
```

Switch status to “Running”:

```
curl localhost:8197/state -X PUT -d "RUNNING" -H "Content-Type:text/plain"
```

Get logs:

```
curl localhost:8197/run-log -H "Content-Type:text/plain"
```

Shut down:

```
curl localhost:8197/state -X PUT -d "SHUTDOWN" -H "Content-Type:text/plain"
```

Development environment

OS: Ubuntu 24.04.1 LTS (Virtual machine)

Host machine: Windows 10

CPU: Intel® Core™ i7-4770S CPU @ 3.10 Ghz (4 cores, all dedicated to VM)

Ram: 32 GB (16 GB dedicate to VM)

Docker version 27.3.1

Docker Compose version v2.29.7

Example runs

Examples of tests failing:

Commit e7a129b4b9f1f54bd228844c43896bfba26d36f0 included adding tests but not implementing the functionality behind the tests:

```
$ docker run --rm --network host test-project
> tests@1.0.0 test
> mocha
Running tests
  1) /state should return INIT when tests are started
  2) /request should return 400 when state is INIT
  3) /run-log should return 400 when state is INIT
  4) should not allow changing state from INIT => PAUSED
  5) should allow changing state from INIT => RUNNING
  6) /request should return 200 when state is RUNNING
  7) /run-log should return 200 when state is RUNNING
  8) should contain 2 lines of logs
  9) should allow changing state from RUNNING => PAUSED
 10) /request should return 400 when state is PAUSED
 11) /run-log should return 400 when state is PAUSED
 12) should allow changing state from PAUSED => RUNNING
 13) should allow changing state from RUNNING => SHUTDOWN
0 passing (92ms)
13 failing
 1) Running tests
    /state should return INIT when tests are started:
    Uncaught AssertionError: expected 'text/html; charset=utf-8' to equal 'text/plain; charset=utf-8'
    + expected - actual
    -text/html; charset=utf-8
    +text/plain; charset=utf-8

    at file:///usr/src/app/test/tests.js:12:48
    at Request.callback (node_modules/superagent/lib/node/index.js:837:12)
    at IncomingMessage.<anonymous> (node_modules/superagent/lib/node/index.js:1078:18)
    at IncomingMessage.emit (node:events:529:35)
    at endReadableNT (node:internal/streams/readable:1400:12)
    at process.processTicksAndRejections (node:internal/process/task_queues:82:21)
 2) Running tests
    /request should return 400 when state is INIT:
    Uncaught AssertionError: expected { Object (_events, _eventsCount, ...) } to have status code 400 but got 200
    + expected - actual
    -200
    +400

    at file:///usr/src/app/test/tests.js:22:29
    at Request.callback (node_modules/superagent/lib/node/index.js:837:12)
    at IncomingMessage.<anonymous> (node_modules/superagent/lib/node/index.js:1078:18)
    at IncomingMessage.emit (node:events:529:35)
    at endReadableNT (node:internal/streams/readable:1400:12)
    at process.processTicksAndRejections (node:internal/process/task_queues:82:21)
 3) Running tests
```

After commit b781e7d5dade0b1d5f05e21f3f213c7044d6cd65 all the tests passed

```
#10 DONE 0.125
1  $ docker run --rm --network host test-project
2  > tests@1.0.0 test
3  > mocha
4  Running tests
5  ✓ /state should return INIT when tests are started
6  ✓ /request should return 400 when state is INIT
7  ✓ /run-log should return 400 when state is INIT
8  ✓ should not allow changing state from INIT => PAUSED
9  ✓ should allow changing state from INIT => RUNNING
10 ✓ /request should return 200 when state is RUNNING (50ms)
11 ✓ /run-log should return 200 when state is RUNNING
12 ✓ should contain 2 lines of logs
13 ✓ should allow changing state from RUNNING => PAUSED
14 ✓ /request should return 400 when state is PAUSED
15 ✓ /run-log should return 400 when state is PAUSED
16 ✓ should allow changing state from PAUSED => RUNNING
17 ✓ should allow changing state from RUNNING => SHUTDOWN
18 13 passing (175ms)
```

Whole pipeline failing and passing:

All 55FinishedBranchesTags

[View analytics](#)[Clear runner caches](#)[New pipeline](#)

Filter pipelines

Q

Show Pipeline ID ▾

Status	Pipeline	Created by	Stages	
✓ Passed 00:04:21 2 hours ago	Remove shutdown button from GUI #687 project ccc32bb6 latest		✓ ✓ ✓	↓
✓ Passed 00:04:08 2 hours ago	Clean up #686 project c24a496c		✓ ✓ ✓	↓
✓ Passed 00:04:22 1 day ago	Clean up #638 project aae4874c		✓ ✓ ✓	↓
✓ Passed 00:04:23 1 day ago	Cleaning up docker-compose.yml #637 project eb651914		✓ ✓ ✓	↓
✓ Passed 00:04:17 1 day ago	Changes to API gateway #631 project a114287b		✓ ✓ ✓	↓
✓ Passed 00:04:07 1 day ago	deployment testing #628 project 32aa08e5		✓ ✓ ✓	↓
✓ Passed 00:04:07 1 day ago	deployment #624 project 1c996842		✓ ✓ ✓	↓
✗ Failed 00:03:44 1 day ago	Deployment #621 project 2af84b94		✓ ✓ ✗	↺ ↓

Reflections

Main learnings and worst difficulties

- Setting up the gitlab runner and properly understanding what the runner does, took some time
- Figuring out how to extend the application with API gateway and trying to find balance between responsibilities of Nginx and API gateway took some time
- Deploying to an external virtual machine caused lot of “trial and error” – though being able to implement a solution where source code change in a development environment can be deployed to public internet with “a click of a button” was definitely worth the struggles
- It is easy to forget to use TDD when there is an issue which’s solution requires trying different approaches.

Amount of effort (hours) used

Estimate: 25-40 hours

Usage of LLM

ChatGPT and Github COPilot were used helping to build the application and the CI/CD pipeline.

The tool were used to:

- Find ideas how to solve certain architectural problems
- Writing boilerplate code
- Help with syntax within the pipeline .yaml file
- Teach me about Gitlab runners